Authentication Decision Log: Session vs JWT

Mehfil Event Planner | Task 1 | 9 November 2026

Decision

We use Laravel Sanctum in session (cookie) mode, with Laravel Breeze login and registration controllers, instead of JWT.

Context

Mehfil Event Planner is a server-rendered Laravel 10 application with two roles, customer and admin. Customers book events and admins manage them from a dashboard. Both use a browser on the same domain.

Session-based authentication

After login the server stores the user in a session and sends an encrypted cookie. Logout destroys the session immediately, so access can be revoked at once. The cookie is HttpOnly, so JavaScript cannot read it, which lowers the risk of token theft through XSS. Laravel adds CSRF protection, session regeneration on login and bcrypt password hashing out of the box. The trade-off is that the server must keep session state, and a pure mobile or third-party API client is less convenient.

JWT

A signed token carries the user identity and role, so the server stays stateless and scales easily across many servers. It suits mobile apps and microservices. However, a JWT cannot be revoked easily before it expires, a stolen token keeps working until then, and storing it in localStorage exposes it to XSS. Refresh-token handling adds extra code and risk.

Why session wins here

The app is one browser-based product on one domain, traffic is small, and instant logout matters for admin accounts. Sanctum stateful mode gives secure cookies and CSRF protection with very little code, and it can later issue API tokens if a mobile app is added. A role middleware allows only admin users on admin routes.

Conclusion

Session-based Sanctum is simpler, safer and a better fit. We would revisit JWT only if a separate mobile client or many independent services need stateless access.